

Programming Project: Combinational Circuit Chip

Michael Frid

May 2026

1 Key Terms & Definitions

Boolean Algebra: A system for manipulating binary values (true/false) using logical operators.

Gate: A logical component that performs a basic Boolean operation on a set of inputs to produce a single result.

Combinational Circuit: A digital circuit where the output depends solely on the set of inputs. It has no memory storage.

Truth Table: A table which lists all possible input combinations and their corresponding outputs for a Boolean function or for a digital circuit.

Karnaugh Map: A graphical way of simplifying Boolean expressions.

2 Reiterate Question

The project asks if it is possible to design a 4-bit ALU that can perform both addition and subtraction operations using only combinational logic circuits.

3 Forecasting

(a) Relationship between addition and subtraction operations

Because $A - B$ is the same as $A + (-B)$, to do subtraction using addition, all that is needed is to find the negative of B . To do so, we can use a two's-complement, which, with signed binary words, is found by inverting all the bits in B (finding B'), and adding one to its result. This way, by controlling the sign of B , the same gate could both add and subtract.

(b) Truth table

A	B	C _{in}	R	C
1	1	1	1	1
1	1	0	0	1
1	0	1	0	1
1	0	0	1	0
0	1	1	0	1
0	1	0	1	0
0	0	1	1	0
0	0	0	0	0

(c) Sum of Product (SOP) Boolean functions

$$R = ABC_{in} + AB'C'_{in} + A'BC'_{in} + A'B'C_{in}$$

$$C = ABC_{in} + ABC'_{in} + AB'C_{in} + A'BC_{in}$$

(d) Simplified Boolean functions

For R:

$$\begin{aligned} R &= ABC_{in} + AB'C'_{in} + A'BC'_{in} + A'B'C_{in} \\ R &= C_{in}(AB + A'B') + C'_{in}(AB' + A'B) \quad [\text{Distributive Law}] \\ R &= C_{in}(A \oplus B)' + C'_{in}(A \oplus B) \quad [\text{XOR/XNOR Property}] \\ R &= C_{in} \oplus (A \oplus B) \quad [\text{XOR Property}] \\ R &= A \oplus B \oplus C_{in} \end{aligned}$$

$$\boxed{R = A \oplus B \oplus C_{in}}$$

For C:

$$\begin{aligned} C &= ABC_{in} + ABC'_{in} + AB'C_{in} + A'BC_{in} \\ C &= AB(C_{in} + C'_{in}) + C_{in}(AB' + A'B) \quad [\text{Distributive Law}] \\ C &= AB + C_{in}(AB' + A'B) \quad [\text{Inverse Property}] \\ C &= AB + C_{in}(A \oplus B) \quad [\text{XOR Property}] \end{aligned}$$

$$\boxed{C = AB + C_{in}(A \oplus B)}$$

(e) LSB stage for the adder circuit

The carry output C_0 becomes the carry input for the next stage of the circuit.

(f) Boolean functions for subtraction

For subtraction, we need B to be inverted, and 1 to be added, which we accomplish by getting the complement of B (B') and setting C_{in} to be initially 1.

$$R = A \oplus B' \oplus C_{in}$$
$$C = AB' + C_{in}(A \oplus B')$$

(g) Merged circuits with control signal

Calling the additional control signal “Sub” we can make $B_{effective} = B \oplus Sub$. The truth table for XOR shows us that if Sub is 0 (addition), nothing happens to B , but if Sub is 1, $B_{effective} = B'$. We can also set C_{in} initial to be equal to Sub, so when Sub is 1, C_{in} is 1, but when Sub is 0, C_{in} is 0, and the circuit can fluidly switch from being an adder to being a subtractor.

4 Hypothesis

If a circuit is constructed using XOR gates for computing the result bit and AND/OR/XOR gates for computing the carry bit, with an additional subtraction indicator bit (Sub), then we will have a 4-bit Arithmetic Logic Unit that can do both addition and subtraction because using the indicator bit can convert a 4-bit adder into a 4-bit subtractor.

5 Step by Step Procedure

6 Implementation Notes

The only major hiccup I ran into was bugs when I was running my code which left me with straight 00000000. I fixed this by running splits to all of my gates, as without them the code wouldn't run properly, but the result was probably not the most efficient implementation. I would hope to come back to the project and I think I could probably refactor the code to be more efficient and have fewer redundancies.

7 Code Repository

gates.h - A header file which defines the logic gates we need for the project: XOR, AND, NOT, and OR.

main.cpp - A file which contains the function definitions of the C circuit and the R circuit, as well as main, which initializes and runs the test cases.

8 Output

My program outputs are stored in files titled `test_case.n.txt`, where n is the number of test case (1-4). Each of the files contains the result of running the test case, as specified in the project requirements.

9 Data Analysis

The test results confirm the correctness of the combinational circuit implementation. Test Case 1 and 2 demonstrates proper 8-bit addition and subtraction with carry propagation across all bit positions. Test Cases 3 and 4 verify the 4-bit ALU functionality: adding $5 + 3$ correctly produces 8 (binary 1000), and subtracting $5 - 3$ correctly produces 2 (binary 0010).

10 Conclusion

Yes, a 4-bit arithmetic logic unit capable of performing both addition and subtraction can be successfully constructed as a combinational circuit. By implementing full adder stages using XOR gates for result computation and AND/OR gates for carry computation, and introducing a control signal that conditionally inverts the second operand via XOR while setting the initial carry input, a single circuit performs both operations. All test cases produced correct results, demonstrating that logic without memory elements or sequential components, can build a functional Adder-Subtractor.